CNN
CONVOLUTIONAL NEURAL NETWORKS

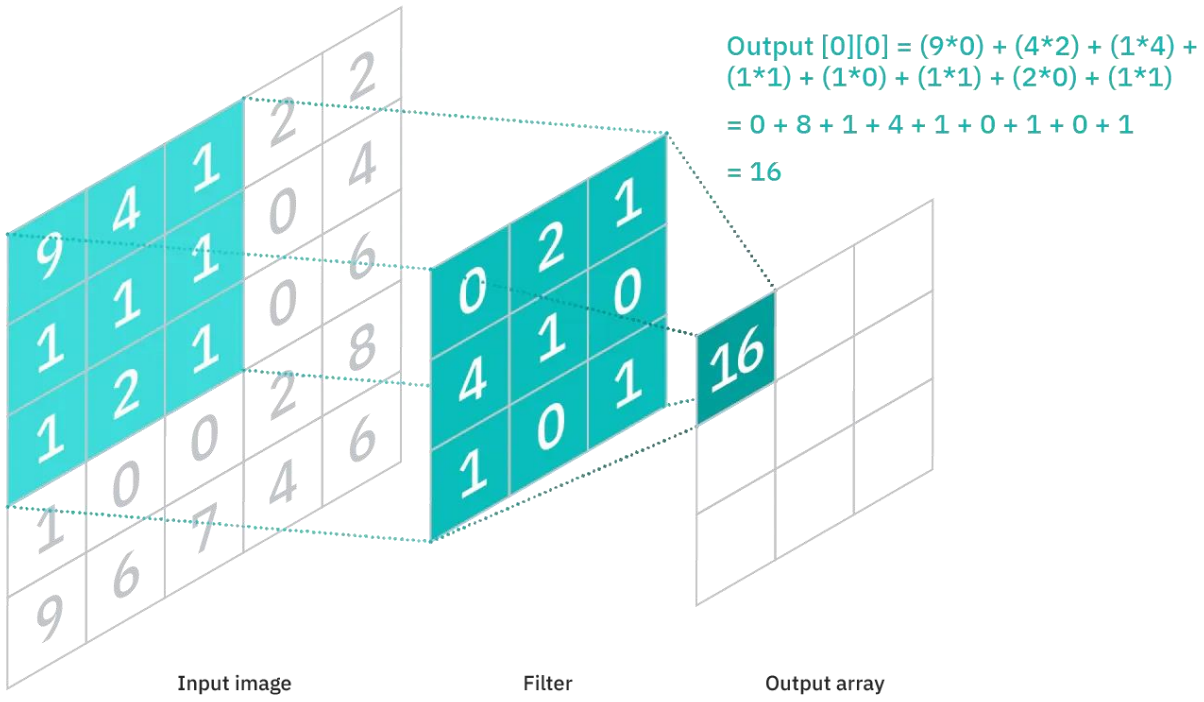
Kourosh GERAYELI

CEDRE 2026

INTRO

Convolution

- A matrix made of numbers, moves it through the data, and takes the sum of products between the data and that matrix.
- This matrix is called kernel or filter.
- The convolved data (the result of the convolution) is called a feature map.



All you need to know now is that the operation simply takes a matrix made of numbers, moves it through the data, and takes the sum of products between the data and that matrix.

This matrix is called kernel or filter.
The convolved data (the result of the convolution) is called a feature map.
brute forcing filters to extract features does not work well with every image, We convolve images with various filters, and these pixels in the feature maps will eventually become the parameters that we will optimize, and in the end, we will find the best filters for our problem.

VGG 16

- A convolutional neural network (CNN) containing **16 learnable layers**.
- Mainly 13 convolution layers and 3 fully connected layers.
- It proved that **depth matters** stacking more layers significantly improves learning, even if each layer is simple.
- VGG16 was designed for the ImageNet dataset, which contains over **14 million images** and **1,000 categories**, ranging from animals to vehicles to household objects.

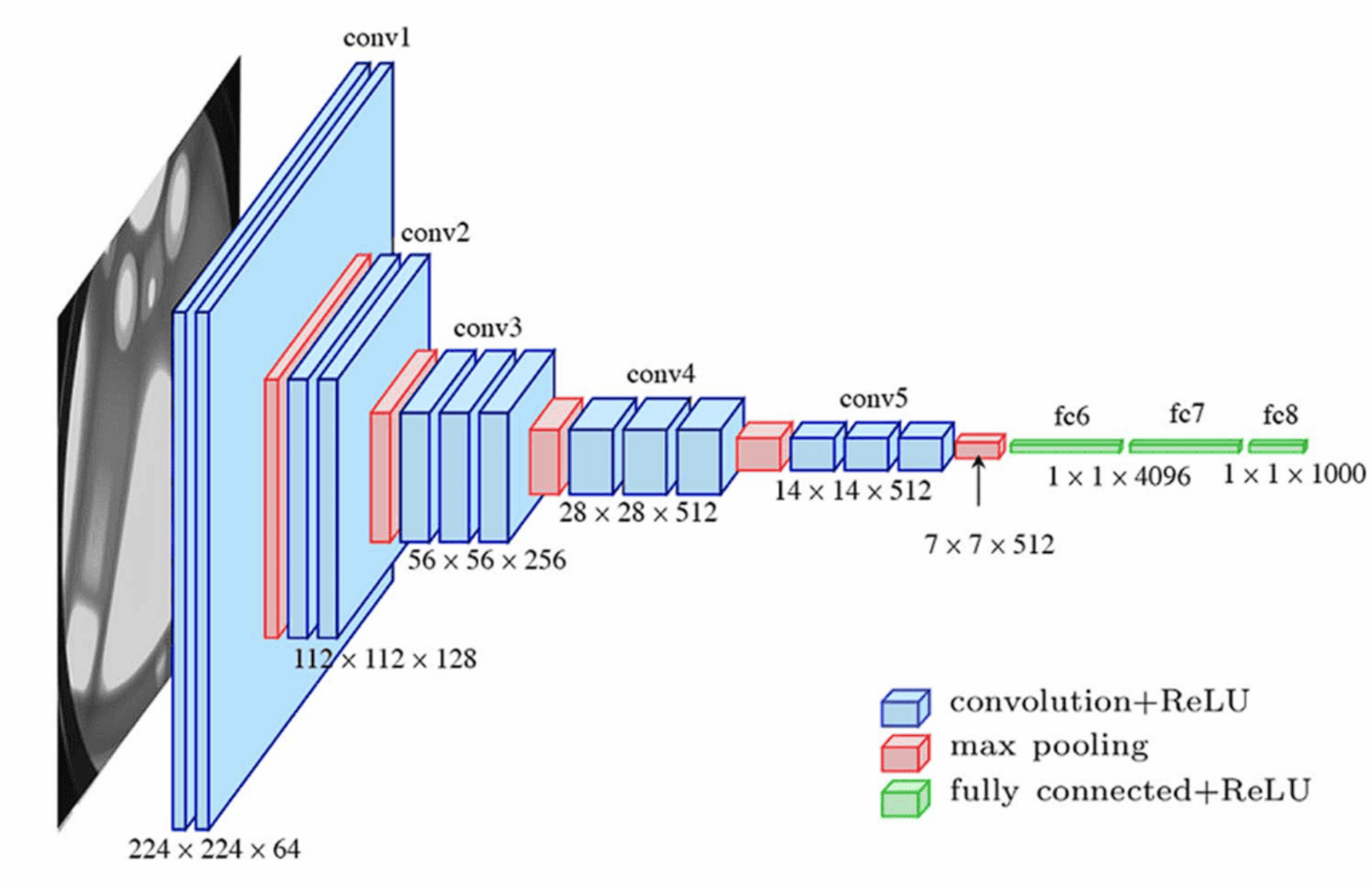
Developed by the Visual Geometry Group (VGG) at the University of Oxford in 2014, this model introduced a simple yet powerful idea: instead of using large filters and complicated designs, stick to **very small 3×3 convolution filters** and build deep networks by stacking them.

Why?

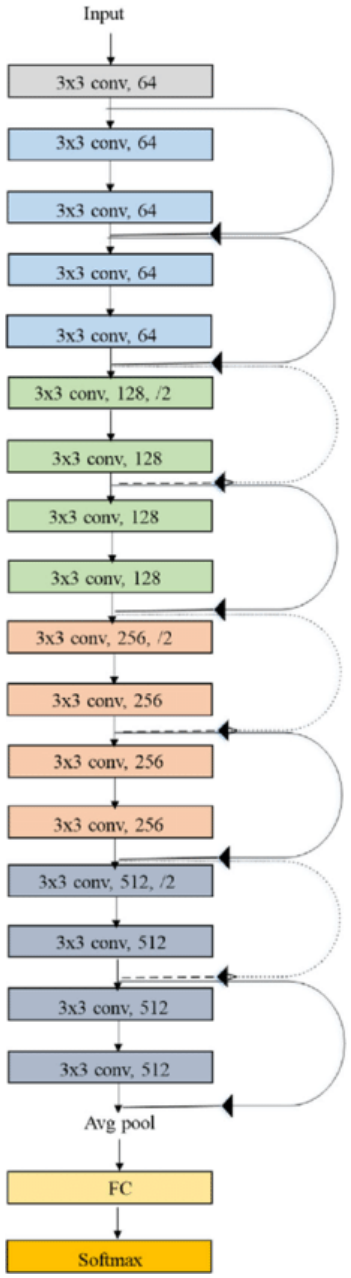
Before VGG16, most CNN models (like AlexNet and ZFNet) used large filters such as **11×11** or **7×7**, which increased parameters and reduced precision. VGG16 introduced a better idea: replace big filters with **multiple small 3×3 filters**. This approach:

- Reduces the number of parameters
- Makes the network deeper
- Extracts more detailed features
- Performs better on large-scale image classification tasks

ARCHITECTURE



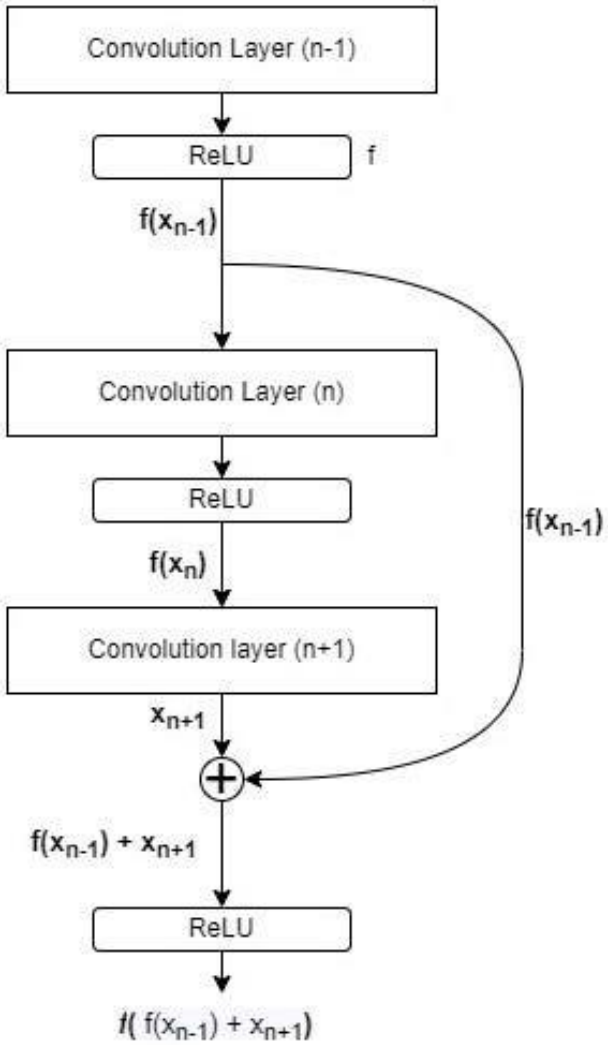
The VGG16 architecture is designed with a clean, uniform, and symmetrical structure that becomes very easy to understand once you look at its block-wise flow. The diagram you provided shows the model from the input layer all the way to the final fully connected layers, clearly depicting how the feature maps shrink in spatial dimension but grow in depth. The model starts with an input image of **224 × 224 × 3**, which then passes through five major convolution blocks. Each block contains multiple **3×3 convolution layers followed by ReLU activation**, and after each block, a **2×2 max pooling layer** reduces the width and height by half. This consistent pattern — Conv → Conv → Pool (or Conv → Conv → Conv → Pool) — is exactly why VGG16 looks so clean and layered in the visual diagram.
In your provided diagram, the architecture progresses as follows: the first block takes the image and produces feature maps of **224 × 224 × 64**, then after the next convolutions, the size becomes **112 × 112 × 128**, followed by **56 × 56 × 256**, then **28 × 28 × 512**, and finally **14 × 14 × 512**, before reaching the smallest feature map of **7 × 7 × 512**.
This step-wise shrinking of spatial size and increasing of depth is exactly what makes CNNs powerful — the early layers learn basic edges and textures, while the deeper layers learn shapes, object parts, and complex structures.
Your diagram beautifully shows this depth progression using blue (Conv+ReLU) and red (MaxPool) blocks, which matches the original VGG paper's architecture.
At the end of the convolutional blocks, the model flattens the **7 × 7 × 512** output into a single long vector and feeds it into three fully connected layers. These layers are clearly shown in green in your diagram: **FC6 (4096 units)**, **FC7 (4096 units)**, and **FC8 (1000 units)** which ends with a **Softmax classifier**. This structure mirrors classic multilayer perceptrons and is the reason VGG16 has such a large parameter count — nearly **120 million parameters** come from these fully connected layers alone.



These shortcuts allow the model to skip layers during forward and backward propagation, helping preserve information and stabilizing during training. ResNet-18 uses simpler blocks to be computationally lighter and faster, while retaining the benefits of residual learning.

But how does this even helps, simply put, if the nth layer is not learning anything even then we won't lose any information, because at $(n+1)^{th}$ we are using the output of $(n-1)^{th}$ layer as well when we move forward and then applying activation on this sum.
Thus we are enabling the network to skip one ReLU activation in between if it does not provide any useful information or provides no information at all i.e, 0, and the network will be using the previous information, thus maintaining consistent performance. If anyways both the layers are providing significant information, thus having previous information will anyways boost the performance.

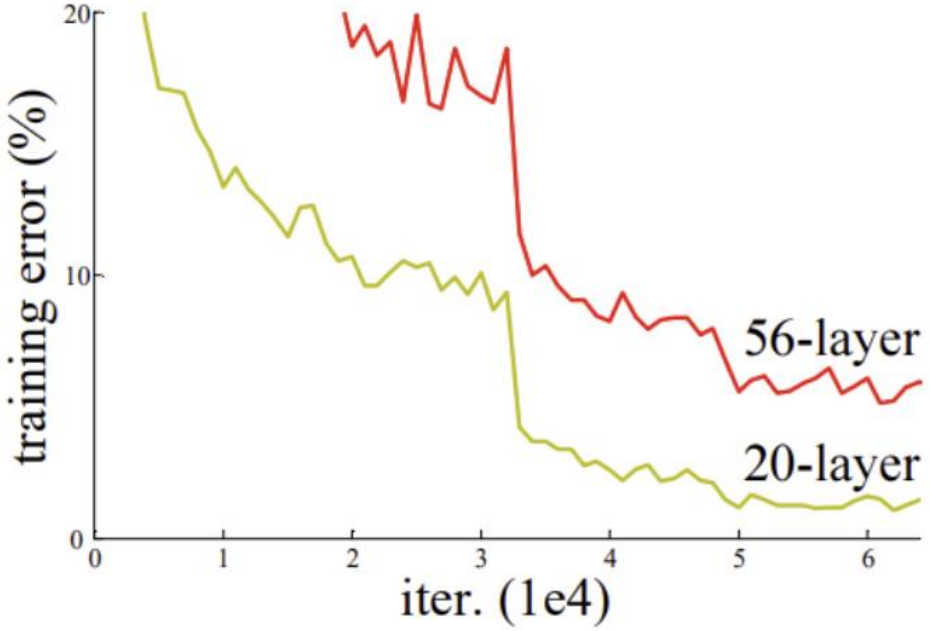
Residual Network

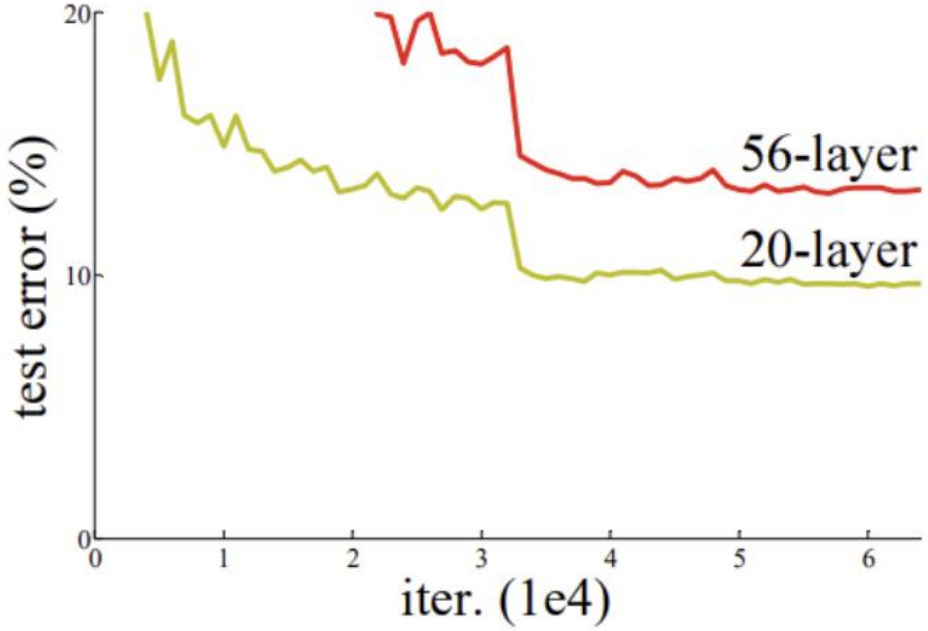


Skip connection

RESNET

We think that the deeper the neural network, the better the performance, but when researchers experimented with deeper neural nets, it was found that adding more layers to a deep network does not always add up to its performance but rather decreases it, which was due to vanishing gradients in very deep neural networks.





NOTEBOOK 3.RESNET

APPENDIX

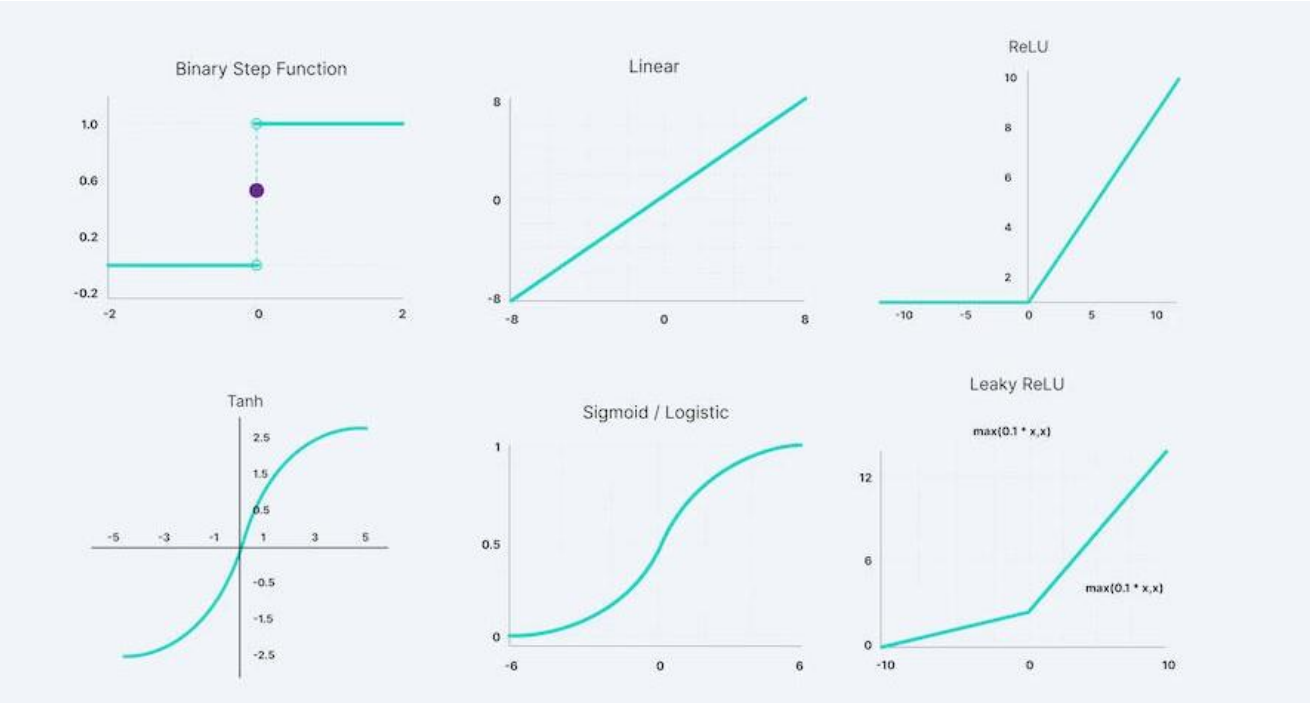
Some definitions

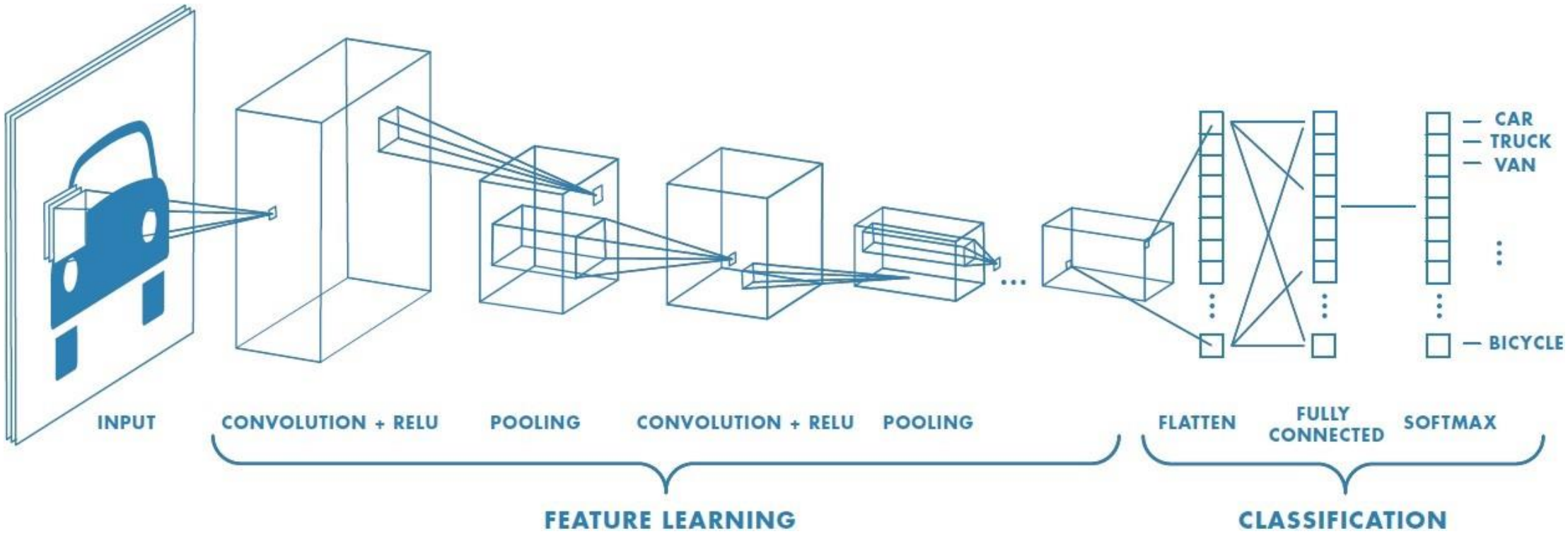
- Convolutional layer
  - Kernel : small matrices that slide over the input image
  - Stride : the step size with which the filter moves across the input image
  - Padding : Adding extra pixels

- Activation function : helps the network learn nonlinear relationships between the features in the image
  - Relu
  - sigmoid
  - Softmax
  - TanH

- Fully connected layers: last layer, their inputs correspond to the flattened one-dimensional matrix generated by the last pooling layer.

- Pooling : getting the most important element in the feature map
  - Average
  - Max pooling
  - weighted pooling
  - L2 pooling




CAR
TRUCK
VAN
BICYCLE
INPUT
CONVOLUTION + RELU
POOLING
CONVOLUTION + RELU
POOLING
FLATTEN
FULLY CONNECTED
SOFTMAX
FEATURE LEARNING
CLASSIFICATION

WHY VGG16 USES ONLY 3×3 FILTERS?

- VGG16 strictly uses **3×3 convolutional filters** throughout its architecture because they are small enough to capture fine details in images while being computationally efficient. By stacking multiple 3×3 convolutions, the network effectively increases the receptive field (the area of the image each neuron sees) without dramatically increasing the number of parameters. For example, two consecutive 3×3 convolutions have the same receptive field as a single 5×5 convolution, but with fewer parameters and more non-linearities, which allows the network to learn more complex features. This design choice makes VGG16 both deep and efficient in capturing spatial hierarchies in images.